

Rapport 2 OCR

Zachary COATES - Manon BIÈRE - Norah Beillevert

decembre 2025



Table des matières

1 Introduction générale **5**
1.1 Contexte et motivations du projet 5
1.2 Problématique générale 5
1.3 Choix d'une approche modulaire et bas niveau 5

Traitement de l'image **7**

2 Architecture **7**
2.1 Vue d'ensemble du pipeline 7
2.2 Architecture modulaire 7
2.3 Organisation des modules 7

3 Prétraitement de l'image **8**
3.1 Chargement et représentation de l'image 8
3.2 Conversion en niveaux de gris 8
3.3 Amélioration du contraste 9
3.4 Binarisation adaptative 10
3.5 Suppression du bruit 11
3.6 Sauvegarde et validation 11
3.7 Impact sur les étapes suivantes 11

4 Détection des contours et des composantes connexes **11**
4.1 Objectif de l'étape 11
4.2 Notion de connexité 12
4.3 Détection des composantes connexes 12
4.4 Extraction des boîtes englobantes 12
4.5 Filtrage des composantes 12
4.6 Visualisation et validation 13
4.7 Limites de l'approche 13
4.8 Transition 13

5 Détection et segmentation de la liste de mots **13**
5.1 Problématique spécifique de la liste de mots 13
5.2 Hypothèses structurelles 14
5.3 Utilisation des bounding boxes des lettres 14
5.4 Tri préalable des lettres 14
5.5 Critère de séparation entre les mots 14

5.6 Regroupement des lettres en mots 15
5.7 Ordonnancement horizontal des lettres 15
5.8 Extraction et sauvegarde des lettres 16
5.9 Analyse des limites 16

6 Détection des contours et composantes connexes 17
6.1 Position du module dans le pipeline 17
6.2 Définition d'un contour dans notre approche 17
6.3 Choix de l'algorithme : le flood-fill 17
6.4 Marquage des pixels visités 18
6.5 Calcul de la bounding box 18
6.6 Filtrage des faux contours 19
6.7 Visualisation des contours 20

7 Détection et reconstruction de la grille de lettres 20
7.1 Pourquoi la grille est le cœur du problème 20
7.2 Rejet des approches géométriques classiques 21
7.3 Changement de paradigme 21
7.4 Définition d'une lettre candidate de la grille 21
7.5 Collecte des lettres de la grille 22
7.6 Tri vertical : préparer la reconstruction 23
7.7 Estimation de la hauteur moyenne 23
7.8 Regroupement des lettres en lignes 23
7.9 Création dynamique des lignes 24
7.10 Tri horizontal au sein des lignes 24
7.11 Pourquoi aucune largeur de cellule n'est utilisée 24
7.12 Sauvegarde des cellules 25

8 Correction de l'orientation de l'image 25
8.1 Motivation 25
8.2 Rotation manuelle 25
8.3 Transformation géométrique 25
8.4 Effets sur le pipeline 26

9 Détection et traitement de la liste de mots 26
9.1 Positionnement de la zone des mots 26
9.2 Différences structurelles avec la grille 26
9.3 Regroupement en mots 26
9.4 Sauvegarde structurée 27

10 Extraction et sauvegarde des lettres **27**
10.1 Découpage des lettres 27
10.2 Normalisation minimale 27
10.3 Organisation sur disque 27

11 Conclusion **28**
11.1 Contexte du projet 29
11.2 Objectifs de ma contribution 29
11.3 Importance et enjeux 29

12 Analyse approfondie des images fournies **30**
12.1 Caractéristiques et diversité du jeu de données 30

13 Pipeline de prétraitement des images **30**

14 Entraînement et stratégie d'apprentissage **31**

15 Analyse des erreurs et ambiguïtés **31**

16 Validation lexicale **32**

17 Bilan des contributions **32**

18 Introduction **32**

19 Contexte et état initial **32**

20 Fonctionnement du solveur **33**
20.1 Conversion de l'image en grille 33
20.2 Normalisation du mot recherché 33
20.3 Recherche du mot 33
20.4 Affichage et gestion mémoire 33

21 Modularisation avec `solver.h` **34**

22 Impact sur le projet global **34**

23 Perspectives **34**

24 Conclusion **34**

25 État Initial et Défis Rencontrés **35**
25.1 Une Architecture Fragmentée 35
25.2 Problèmes d'Intégration 35
25.3 Limitations Techniques Initiales 36
25.4 Objectifs de Transformation 36

26 Réalisations Majeures **37**
26.1 Création d'une Interface Graphique Complète 37
26.2 Système de Traitement par Étapes 38
26.3 Intégration des Composants Existants 39
26.4 Robustesse et Maintenabilité 41

27 Fonctionnalités Clés Implémentées **42**

28 Points Forts du Système Actuel **43**

29 Pistes d'Amélioration et Évolutions Possibles **43**
29.1 Améliorations de l'Expérience Utilisateur 43
29.2 Optimisations de Performance et Gestion de Mémoire 44
29.3 Fonctionnalités Manquantes à Implémenter 44
29.4 Améliorations des Algorithmes de Traitement 45
29.5 Renforcement de l'Intégration et des Tests 45
29.6 Évolutions Architecturales 46

30 Conclusion **46**

Annexe — Étapes du traitement de l'image **47**

1 Introduction générale

1.1 Contexte et motivations du projet

La reconnaissance automatique de caractères (*Optical Character Recognition*, OCR) est un domaine clé de l'informatique, combinant traitement d'image, intelligence artificielle et algorithmique. Elle vise à transformer une information visuelle contenant du texte en une représentation exploitable par un programme.

Ce projet applique les principes de l'OCR à un cas particulier : l'analyse automatique d'une image contenant une grille de lettres et une liste de mots associée, dans le but de reconnaître les caractères et de résoudre le jeu correspondant. L'objectif n'est pas d'utiliser des solutions clés en main, mais de construire manuellement l'ensemble du pipeline afin de comprendre les mécanismes fondamentaux de chaque étape.

Le système repose sur trois composantes principales :

— le traitement d'image pour extraire les lettres et la structure de la grille ;
— la reconnaissance des caractères à l'aide d'un réseau de neurones ;
— la résolution algorithmique du jeu à partir des données reconnues.

1.2 Problématique générale

L'image traitée ne correspond pas à un simple document textuel. Elle contient à la fois une grille de lettres, une liste de mots adjacente, ainsi que des imperfections visuelles telles que du bruit ou des variations d'éclairage.

La problématique centrale du projet est la suivante :

> Comment analyser automatiquement une image structurée complexe afin d'en extraire les lettres, de les reconnaître correctement et de résoudre le problème logique associé, sans connaissance préalable précise de son organisation ?

1.3 Choix d'une approche modulaire et bas niveau

Le projet adopte une approche modulaire, dans laquelle chaque étape du pipeline est clairement séparée. Les images sont manipulées à un niveau bas, directement au niveau du pixel, sans recours à des bibliothèques de vision artificielle avancées.

Cette approche permet un contrôle précis des données et une observation claire des résultats intermédiaires, au prix de l'implémentation manuelle de nombreuses opérations classiques. Ce choix est assumé et s'inscrit dans une démarche pédagogique visant à relier traitement d'image, intelligence artificielle et algorithmique de résolution au sein d'un système cohérent.

Présentation des membres et de leurs missions

Zachary COATES :

Je m'appelle Zachary et je suis en charge de la partie réseau de neurones du projet, dont l'objectif est la reconnaissance automatique des lettres. J'ai d'abord travaillé sur des réseaux simples afin de comprendre leur fonctionnement, notamment à travers un modèle capable de reconnaître des caractères.

Manon BIERE :

Je m'appelle Manon et je suis responsable du solver ainsi que de la reconstruction logique de la grille et de la liste de mots à partir des résultats fournis par la reconnaissance de caractères. Mon travail intervient donc dans la phase finale du projet.

J'ai dans un premier temps développé un solver en langage C, capable d'analyser une grille de mots cachés et de localiser un mot donné. Ce module constitue le cœur algorithmique de la résolution et servira ensuite à exploiter les données issues du réseau de neurones.

Norah BEILLEVERT :

Je m'appelle Norah et je suis en charge de la détection et du prétraitement d'image. Mon rôle consiste à transformer une image brute en éléments exploitables : détection de la grille, de la liste de mots et extraction des lettres individuelles.

Cette étape est essentielle, car la qualité du prétraitement conditionne directement les performances du réseau de neurones et du solver. Mon objectif est donc de fournir des données aussi propres et structurées que possible pour les étapes suivantes du pipeline.

Traitement de l'image

2 Architecture

2.1 Vue d'ensemble du pipeline

Le projet suit un pipeline de traitement d'image simple et linéaire, dans lequel l'image brute est progressivement transformée en données exploitables.

> Image brute → Prétraitement → Détection des contours → Segmentation → Extraction des lettres

Chaque étape est isolée dans un module distinct afin de garantir une organisation claire et un débogage facilité.

2.2 Architecture modulaire

L'architecture repose sur une séparation stricte des responsabilités. Chaque module correspond à une étape précise du pipeline et peut être testé indépendamment. Cette organisation limite la propagation des erreurs et facilite l'ajustement des paramètres.

2.3 Organisation des modules

2.3.1 `main.c`

Le fichier `main.c` constitue le point d'entrée du programme. Il se limite au chargement de l'image et à l'appel séquentiel des différentes étapes du pipeline.

2.3.2 `Traitement de l'image :`

Cette partie regroupe l'ensemble des modules responsables de l'analyse et de la segmentation de l'image.

`image_utils.c` Fournit les fonctions bas niveau de manipulation des images (accès aux pixels, conversions et sauvegardes).

`image_processing.c` Implémente le prétraitement : niveaux de gris, amélioration du contraste et binarisation afin de faire ressortir les lettres.

`contour_detection.c` Détecte les composantes connexes de l'image binarisée et extrait les bounding boxes des régions pertinentes.

`grid_detection.c` Analyse la structure globale de l'image, distingue grille et mots, et organise les lettres avant leur sauvegarde.

3 Prétraitement de l'image

Le prétraitement constitue une étape fondamentale de tout système de reconnaissance optique de caractères. Une image brute contient de nombreuses informations inutiles, voire nuisibles, pour la détection des lettres : variations d'éclairage, bruit de numérisation, artefacts visuels ou informations de couleur redondantes.

L'objectif du prétraitement n'est donc pas d'améliorer l'image visuellement, mais de transformer progressivement une image hétérogène en une représentation exploitable, stable et adaptée aux algorithmes de segmentation. Sans cette étape, les traitements ultérieurs produiraient un nombre important de faux contours, des bounding boxes incorrectes et une segmentation instable. Le prétraitement agit ainsi comme un véritable filtre de stabilisation du pipeline.

Cette section détaille les opérations de prétraitement introduites dans l'architecture générale du projet, en explicitant les choix algorithmiques, les formulations mathématiques et leur implémentation.

3.1 Chargement et représentation de l'image

Le chargement de l'image est assuré à l'aide de la bibliothèque SDL2, qui permet de lire les formats courants tels que BMP, PNG et JPEG. L'image est initialement stockée sous la forme d'une surface SDL contenant les dimensions et les données pixel.

À ce stade, l'image est représentée en couleur selon le modèle RGB. Cette représentation, bien qu'adaptée à l'affichage, est peu pertinente pour le traitement, car elle multiplie la quantité d'information sans apporter de valeur pour la détection des lettres, qui repose principalement sur le contraste et la géométrie.

3.2 Conversion en niveaux de gris

La première transformation consiste à convertir l'image couleur en une image en niveaux de gris. Les algorithmes de segmentation et de détection utilisés ne nécessitent aucune information chromatique.

Cette conversion permet :
— de réduire la complexité mémoire ;
— de simplifier les calculs ;
— d'unifier le traitement quelle que soit la couleur originale de l'image.

Deux formulations équivalentes sont couramment utilisées. Conceptuellement, la conversion repose sur la combinaison pondérée des composantes RGB :

$$I = 0.299R + 0.587G + 0.114B$$

Ces coefficients reflètent la sensibilité relative de l'œil humain aux différentes couleurs et produisent une image en niveaux de gris cohérente et contrastée.

Dans l'implémentation du projet, une moyenne arithmétique simple est utilisée :

$$G = \frac{R + G + B}{3}$$

Cette approximation, suffisante dans le cadre du projet, offre un bon compromis entre simplicité et efficacité. Le traitement est effectué dès le chargement de l'image, comme illustré dans le code :

```
img->pixels[y*w + x] = (r + g + b) / 3;
```

3.3 Amélioration du contraste

Une image en niveaux de gris peut présenter un contraste insuffisant, notamment en présence d'un éclairage non homogène, de zones gris clair mal différenciées ou d'une luminosité globale déséquilibrée. Ces défauts compliquent fortement la binarisation.

Afin d'y remédier, une amélioration du contraste par étirement linéaire des intensités est appliquée. Soient :

$$I_{\min} = \min(I), \quad I_{\max} = \max(I)$$

Chaque pixel est transformé selon :

$$I' = \frac{(I - I_{\min}) \cdot 255}{I_{\max} - I_{\min}}$$

Cette transformation maximise l'occupation de l'intervalle $[0, 255]$ sans introduire de distorsion locale excessive. Elle facilite ainsi la séparation ultérieure entre le fond et les lettres.

Ce choix est motivé par :

— sa simplicité algorithmique ;
— l'absence de paramètres complexes ;
— sa robustesse sur des images de qualité variable.

Dans l'implémentation, une protection est ajoutée afin d'éviter les divisions instables :

```
if (max - min < 15) return;
```

3.4 Binarisation adaptative

La binarisation transforme l'image en niveaux de gris en une image binaire composée uniquement de pixels noirs et blancs, dans laquelle les lettres doivent apparaître comme des régions distinctes.

Une binarisation globale à seuil fixe échoue dès que l'éclairage est non uniforme, que des ombres sont présentes ou que le fond n'est pas parfaitement homogène. Une binarisation adaptative locale est donc utilisée.

Pour chaque pixel (x, y), un seuil est calculé à partir de la moyenne des intensités dans une fenêtre locale centrée sur ce pixel :

$$T(x, y) = \frac{1}{N} \sum_{(i,j) \in \mathcal{W}} I(i, j) - C$$

où :
— $\mathcal{W}$ représente la fenêtre locale ;
— N est le nombre de pixels qu'elle contient ;
— C est une constante empirique.

La décision binaire est alors :

$$B(x, y) = \begin{cases} 0 & \text{si } I(x, y) < T(x, y) \\ 255 & \text{sinon} \end{cases}$$

Dans notre implémentation, la taille de la fenêtre est fixée à $w = 5$ et la constante à $C = 3$. Ces valeurs offrent un compromis satisfaisant entre la préservation des traits fins des lettres et la robustesse face aux variations d'éclairage.

Cette logique correspond directement au code suivant :

```
int threshold = (sum / count) - 3;
tmp[y*w + x] = (img->pixels[y*w + x] < threshold) ? 0 : 255;
```

3.5 Suppression du bruit

Après binarisation, l'image peut encore contenir du bruit sous forme de pixels noirs isolés ou de petites régions parasites, dus notamment à la texture du papier, aux artefacts de numérisation ou aux irrégularités d'éclairage.

Une opération de *despeckle* est appliquée. Pour chaque pixel noir, le nombre de voisins noirs dans un voisinage 3×3 est compté. Soit n ce nombre.

La règle de suppression est définie comme suit :

$$\text{si } n \leq 2 \Rightarrow \text{le pixel est supprimé}$$

Cette approche présente plusieurs avantages :
— elle élimine efficacement les points isolés ;
— elle préserve les traits continus des lettres ;
— elle reste très peu coûteuse en calcul.

En complément, les régions résiduelles trop petites sont filtrées lors de la détection des composantes connexes, ce qui permet d'éviter l'utilisation d'opérations morphologiques plus complexes.

3.6 Sauvegarde et validation

L'image binaire obtenue est systématiquement sauvegardée sur disque. Cette étape permet une validation visuelle du prétraitement, facilite le débogage et rend le comportement des algorithmes observable avant les étapes suivantes du pipeline.

3.7 Impact sur les étapes suivantes

Le prétraitement conditionne directement :
— la qualité de la détection de contours ;
— la précision des bounding boxes ;
— la stabilité de la segmentation grille et mots.

Une binarisation ou un contraste mal maîtrisé se propage à l'ensemble du système et dégrade significativement les performances globales.

4 Détection des contours et des composantes connexes

4.1 Objectif de l'étape

Après le prétraitement et la binarisation, l'image peut être considérée comme une matrice binaire dans laquelle les pixels noirs correspondent à des structures d'intérêt

potentielles.

L'objectif de cette étape est de regrouper ces pixels en entités cohérentes, appelées *composantes connexes*, puis de caractériser chacune d'elles par une représentation géométrique simple. Cette transition marque le passage du niveau pixel au niveau objet.

4.2 Notion de connexité

Deux définitions classiques de la connexité existent en traitement d'image : la connexité 4 et la connexité 8. Dans ce projet, la connexité 4 a été retenue afin de limiter les connexions artificielles dues au bruit et de conserver des séparations nettes entre les formes proches.

4.3 Détection des composantes connexes

La détection repose sur un parcours exhaustif de l'image. Lorsqu'un pixel noir non visité est rencontré, un algorithme de *flood fill* est déclenché afin de marquer l'ensemble des pixels noirs connexes appartenant à la même composante.

Cette approche garantit que chaque pixel est visité une seule fois et permet d'identifier l'intégralité des régions connexes de l'image.

4.4 Extraction des boîtes englobantes

Pour chaque composante détectée, une boîte englobante (*bounding box*) est calculée. Elle est définie par les coordonnées minimales et maximales de la composante ainsi que par sa largeur et sa hauteur.

Les boîtes englobantes constituent une représentation compacte permettant :

— de localiser précisément chaque forme ;
— de comparer les tailles relatives des objets ;
— de préparer le découpage ultérieur des lettres.

4.5 Filtrage des composantes

Toutes les composantes détectées ne correspondent pas à des lettres. Un filtrage géométrique est donc appliqué à partir de critères simples :

— largeur et hauteur minimales ;
— largeur et hauteur maximales ;
— rapport largeur / hauteur.

Une attention particulière est portée aux lettres fines, telles que le caractère *I*. Les seuils sont volontairement permissifs afin d'éviter la suppression de lettres valides, quitte à conserver quelques faux positifs.

4.6 Visualisation et validation

Afin de faciliter le débogage, les boîtes englobantes conservées sont affichées sur l'image sous forme de rectangles. Cette visualisation permet de comprendre rapidement les erreurs de détection et d'ajuster les paramètres de filtrage.

4.7 Limites de l'approche

La méthode reste dépendante de la qualité de la binarisation et peut rencontrer des difficultés lorsque des lettres sont collées ou fortement dégradées. Toutefois, ces limitations restent acceptables dans le cadre du projet, compte tenu de la structure régulière des images traitées.

4.8 Transition

À l'issue de cette étape, l'image est représentée comme un ensemble structuré de régions géométriques. Cette représentation permet d'aborder la détection des zones fonctionnelles de l'image, en particulier la séparation entre la grille de lettres et la liste de mots.

5 Détection et segmentation de la liste de mots

5.1 Problématique spécifique de la liste de mots

Contrairement à la grille, la liste de mots ne présente pas de structure matricielle explicite. Les mots sont disposés de manière essentiellement linéaire, avec :

— une orientation horizontale dominante ;
— des espacements variables entre les lettres ;
— des séparations irrégulières entre les mots.

Cette absence de régularité impose une approche plus heuristique que celle utilisée pour la grille.

5.2 Hypothèses structurelles

5.2.1 Alignement horizontal

L'algorithme repose sur l'hypothèse que :

> Les lettres appartenant à un même mot partagent une coordonnée verticale proche.

Cette hypothèse est raisonnable dans le contexte de documents imprimés ou manuscrits soignés.

5.2.2 Séparation verticale entre les mots

Les mots successifs sont généralement séparés par un espace vertical plus important que celui séparant deux lettres d'un même mot.

Cette observation est exploitée pour segmenter la liste.

5.3 Utilisation des bounding boxes des lettres

Les bounding boxes extraites précédemment sont réutilisées.

Chaque lettre est représentée par :

$$b_i = (x_i, y_i, w_i, h_i)$$

où :

— (x_i, y_i) est le coin supérieur gauche ;

— w_i et h_i sont la largeur et la hauteur.

La segmentation des mots est donc un problème de regroupement de ces boîtes.

5.4 Tri préalable des lettres

5.4.1 Tri par coordonnée verticale

Les bounding boxes sont triées par y_i croissant.

Ce tri permet de parcourir les lettres de haut en bas, ce qui correspond à l'ordre naturel de lecture.

5.5 Critère de séparation entre les mots

5.5.1 Distance verticale

Soit deux lettres consécutives b_i et b_{i+1} après tri.

On définit la distance verticale :

$$\Delta y = |y_{i+1} - y_i|$$

Si Δy dépasse un seuil, on considère qu'un nouveau mot commence.

5.5.2 Choix du seuil

Le seuil est choisi empiriquement :

$$\Delta y > \beta \cdot \bar{h}$$

où :
— $\bar{h}$ est la hauteur moyenne des lettres ;
— β est un coefficient compris typiquement entre 1.5 et 2.
Ce choix rend le critère invariant à l'échelle de l'image.

5.6 Regroupement des lettres en mots

5.6.1 Construction incrémentale

L'algorithme parcourt les lettres une par une :
— si la lettre est proche verticalement de la précédente, elle est ajoutée au mot courant ;
— sinon, un nouveau mot est créé.
Cette approche incrémentale est simple, mais efficace.

5.6.2 Gestion des erreurs locales

Des erreurs locales (lettre mal détectée, bruit) peuvent créer des ruptures artificielles.

Pour limiter cet effet :
— un mot contenant une seule lettre peut être fusionné avec le suivant ;
— des seuils adaptatifs peuvent être utilisés.

5.7 Ordonnancement horizontal des lettres

5.7.1 Tri interne des lettres d'un mot

À l'intérieur d'un mot, les lettres sont triées par coordonnée x croissante.
Cela garantit l'ordre correct pour :

— la sauvegarde des images ;
— la reconnaissance OCR ultérieure ;
— la reconstruction textuelle.

5.7.2 Hypothèse de non-chevauchement

On suppose que deux lettres d'un même mot ne se chevauchent pas horizontalement.

Cette hypothèse est raisonnable après le prétraitement et la détection de contours.

5.8 Extraction et sauvegarde des lettres

5.8.1 Découpage individuel

Chaque lettre est découpée à partir de son bounding box.

Un padding est ajouté autour de la lettre pour :
— éviter les coupures trop serrées ;
— standardiser les entrées pour l'OCR ;
— réduire l'impact des erreurs de segmentation.

5.8.2 Organisation des dossiers

Les lettres sont sauvegardées selon la hiérarchie suivante :

```
out/
 words/
     word_001/
        letter_000.bmp
        letter_001.bmp
     word_002/
```

Cette organisation reflète directement la structure logique détectée.

5.9 Analyse des limites

5.9.1 Cas des mots inclinés

Si les mots ne sont pas parfaitement horizontaux, le critère basé sur Δy peut échouer.

Ce cas pourrait être traité par :
— une estimation de l'orientation dominante ;

— une projection le long de cette orientation.

Cette approche complète naturellement la détection de la grille et prépare les données pour la reconnaissance finale.

6 Détection des contours et composantes connexes

6.1 Position du module dans le pipeline

Après le prétraitement, l'image est binaire et débarrassée d'une grande partie de son bruit. Cependant, elle reste une simple matrice de pixels noirs et blancs.

Le rôle du module de détection de contours est de transformer cette matrice en une représentation structurée :

— des régions cohérentes ;

— des lettres individuelles ;

— des zones exploitables pour la segmentation.

Cette étape constitue le pont entre le traitement bas niveau (pixels) et la compréhension structurelle de l'image.

6.2 Définition d'un contour dans notre approche

Contrairement à des méthodes basées sur les gradients (Sobel, Canny, etc.), un contour est ici défini comme :

> Une composante connexe de pixels noirs dans une image binaire.

Autrement dit, deux pixels appartiennent au même contour s'ils sont :

— noirs ;

— adjacents par voisinage direct.

Cette définition est volontairement simple et parfaitement adaptée à la reconnaissance de lettres imprimées.

6.3 Choix de l'algorithme : le flood-fill

6.3.1 Principe général

L'algorithme de flood-fill permet de parcourir l'ensemble des pixels appartenant à une même composante connexe.

Son fonctionnement est analogue à un remplissage de peinture :

— on démarre depuis un pixel noir non visité ;

— on explore récursivement ses voisins ;

— on marque tous les pixels atteignables.

6.3.2 Voisinage utilisé

Dans ce projet, un voisinage à 4 connexions est utilisé :

$$(x+1, y),\ (x-1, y),\ (x, y+1),\ (x, y-1)$$

Ce choix limite les connexions diagonales parasites et stabilise la détection.

6.3.3 Avantages de ce choix

— implémentation simple ;
— aucun calcul de gradient ;
— robuste après une bonne binarisation.

6.4 Marquage des pixels visités

Afin d'éviter de parcourir plusieurs fois la même zone, un tableau booléen `visited` est utilisé.

Chaque pixel est associé à un état :
— `false` : pixel non encore exploré ;
— `true` : pixel déjà traité.

Mathématiquement, cela permet de garantir une complexité linéaire :

$$O(w \cdot h)$$

où w et h sont les dimensions de l'image.

6.5 Calcul de la bounding box

6.5.1 Objectif

Pour chaque composante connexe détectée, on souhaite calculer un rectangle englobant minimal (bounding box).

Ce rectangle est défini par :

$$(x_{\min}, y_{\min}, x_{\max}, y_{\max})$$

6.5.2 Mise à jour incrémentale

À chaque pixel exploré, on met à jour :

$$\begin{aligned} x_{\min} &= \min(x_{\min}, x) \\ y_{\min} &= \min(y_{\min}, y) \\ x_{\max} &= \max(x_{\max}, x) \\ y_{\max} &= \max(y_{\max}, y) \end{aligned}$$

La largeur et la hauteur sont ensuite calculées par :

$$w = x_{\max} - x_{\min} + 1 \quad ; \quad h = y_{\max} - y_{\min} + 1$$

6.5.3 Lien avec le code

Cette logique est implémentée directement dans la fonction récursive `flood_fill` :

```
if (x < *minx) *minx = x;
if (y < *miny) *miny = y;
if (x > *maxx) *maxx = x;
if (y > *maxy) *maxy = y;
```

6.6 Filtrage des faux contours

6.6.1 Problème des faux positifs

Même après prétraitement, certaines régions détectées ne correspondent pas à des lettres :

— bruit résiduel ;
— artefacts ;
— éléments graphiques.

Il est donc nécessaire de filtrer les bounding boxes.

6.6.2 Critères géométriques

Une bounding box est considérée valide si :

— sa largeur est suffisante ;
— sa hauteur est suffisante ;
— son rapport largeur/hauteur est cohérent.

Formellement :

$$r = \frac{w}{h}$$

La boîte est acceptée si :

$$r_{\min} \leq r \leq r_{\max}$$

6.6.3 Justification du ratio

Les lettres imprimées possèdent des proportions relativement stables. Ce critère permet notamment :

— d'accepter les lettres fines comme I ;
— d'éliminer les lignes parasites ;
— d'éviter les zones trop larges.

6.6.4 Implémentation

```
if (bw >= 2 && bh >= 8 && bw < w/2 && bh < h/2)
    boxes[n++] = (BoundingBox){minx, miny, bw, bh};
```

6.7 Visualisation des contours

Afin de valider visuellement les résultats, les bounding boxes détectées sont dessinées sur l'image finale.

Cette visualisation permet :

— de déboguer les paramètres ;
— d'ajuster les seuils ;
— de comprendre les erreurs de segmentation.

7 Détection et reconstruction de la grille de lettres

7.1 Pourquoi la grille est le cœur du problème

Dans le cadre de ce projet, la grille de lettres constitue l'élément central de l'image. Contrairement à un simple texte linéaire, elle introduit une structure bidimensionnelle :

— organisation en lignes ;
— organisation en colonnes ;
— régularité partielle mais jamais parfaite.

La difficulté principale provient du fait que cette grille n'est pas matérialisée graphiquement. Il n'existe :

— ni lignes tracées ;
— ni bordures ;
— ni cellules visibles.

La grille n'existe donc que par la disposition spatiale des lettres elles-mêmes.

7.2 Rejet des approches géométriques classiques

Une approche naïve consisterait à rechercher :

— des lignes horizontales ;
— des lignes verticales ;
— des intersections régulières.

Ces méthodes sont efficaces pour des grilles imprimées avec des bordures visibles. Cependant, elles échouent totalement dans notre contexte.

Les raisons principales sont :

— absence de traits graphiques ;
— variations d'alignement ;
— lettres manquantes ;
— irrégularités d'espacement.

Toute hypothèse de grille parfaitement régulière conduit donc à des erreurs systématiques.

7.3 Changement de paradigme

Plutôt que de détecter une grille en tant qu'objet, le choix a été fait de :

> Reconstruire la grille à partir des lettres détectées.

Ce changement de paradigme est fondamental. Il permet de s'adapter à des grilles :

— incomplètes ;
— déformées ;
— partiellement occluses.

7.4 Définition d'une lettre candidate de la grille

Toutes les lettres détectées ne peuvent pas appartenir à la grille. Un filtrage spécifique est donc appliqué.

Une lettre est considérée comme appartenant à la grille si :

— elle est entièrement contenue dans la zone grille ;
— sa taille est compatible avec une cellule ;
— son ratio largeur/hauteur est plausible.

7.4.1 Contraintes dimensionnelles

Soit une bounding box de dimensions (w, h).
Les conditions imposées sont :

$$w_{\min} \leq w \leq w_{\max} \quad \text{et} \quad h_{\min} \leq h \leq h_{\max}$$

Ces bornes sont définies empiriquement à partir des images testées.

7.4.2 Filtrage par ratio

Le ratio :

$$r = \frac{w}{h}$$

permet d'éliminer :
— les lignes horizontales parasites ;
— les taches verticales de bruit ;
— les fragments trop allongés.
La condition appliquée est :

$$r_{\min} \leq r \leq r_{\max}$$

7.5 Collecte des lettres de la grille

Après filtrage, toutes les bounding boxes valides sont stockées dans une liste de lettres candidates.
À ce stade :
— l'ordre n'a aucune importance ;
— aucune structure n'est encore imposée ;
— seule la position spatiale compte.
Cette approche volontairement non ordonnée évite toute hypothèse prématurée sur la forme de la grille.

7.6 Tri vertical : préparer la reconstruction

La première étape de structuration consiste à trier les lettres selon leur coordonnée verticale y.

Ce tri repose sur l'hypothèse suivante :

> Les lettres appartenant à une même ligne possèdent des coordonnées verticales proches.

Mathématiquement, on applique :

$$\text{sort}(B, \text{clé} = y)$$

où B est l'ensemble des bounding boxes.

7.7 Estimation de la hauteur moyenne

Afin de définir une tolérance de regroupement, la hauteur moyenne des lettres est calculée :

$$\bar{h} = \frac{1}{N} \sum_{i=1}^{N} h_i$$

Cette valeur sert d'unité de référence pour tous les seuils suivants.

L'avantage principal est l'invariance partielle à l'échelle :

— une image plus grande produit des lettres plus grandes ;

— les seuils s'adaptent automatiquement.

7.8 Regroupement des lettres en lignes

7.8.1 Principe

Chaque lettre est comparée aux lignes déjà existantes.

Une lettre b est intégrée à une ligne L si :

$$|y_b - y_L| < \alpha \cdot \bar{h}$$

où :

— y_b est la coordonnée verticale de la lettre ;

— y_L est la coordonnée moyenne de la ligne ;

— α est un coefficient empirique.

7.8.2 Choix du coefficient

Le coefficient α a été fixé expérimentalement autour de 0.6.
Ce choix permet :
— de tolérer des décalages verticaux ;
— d'accepter des lettres légèrement inclinées ;
— d'éviter les fusions de lignes distinctes.

7.9 Création dynamique des lignes

Si une lettre ne correspond à aucune ligne existante, une nouvelle ligne est créée.
Cette logique garantit que :
— le nombre de lignes n'est jamais imposé ;
— la grille peut être partiellement remplie ;
— les lignes sont découvertes dynamiquement.

7.10 Tri horizontal au sein des lignes

Une fois les lignes construites, chaque ligne est traitée indépendamment.
Les lettres sont triées selon leur coordonnée horizontale x :

$$\text{sort}(L, \text{clé} = x)$$

Ce tri permet de :
— reconstituer l'ordre des colonnes ;
— reconstruire implicitement la grille ;
— tolérer des espacements irréguliers.

7.11 Pourquoi aucune largeur de cellule n'est utilisée

Contrairement aux grilles géométriques classiques, aucune largeur de cellule n'est imposée.
Les avantages sont multiples :
— robustesse aux déformations ;
— indépendance vis-à-vis des marges ;
— tolérance aux lettres manquantes.
La grille est ainsi définie par l'ordre des lettres, et non par un quadrillage fixe.

7.12 Sauvegarde des cellules

Chaque lettre de la grille est ensuite découpée et sauvegardée individuellement. Le nommage suit la convention :

`rXX_cYY.bmp`

où :
— XX est l'indice de ligne ;
— YY est l'indice de colonne.
Cette convention facilite :
— l'exploitation ultérieure ;
— la reconstruction logique ;
— l'entraînement d'un OCR.

8 Correction de l'orientation de l'image

8.1 Motivation

Les images fournies ne sont pas toujours parfaitement alignées. Une légère rotation, même de quelques degrés, suffit à perturber :
— la détection des lignes de la grille ;
— le regroupement vertical des lettres ;
— la segmentation des mots.
Il a donc été nécessaire d'introduire une étape de correction d'orientation.

8.2 Rotation manuelle

Contrairement à des approches automatiques complexes (détection de lignes de Hough, PCA), le choix a été fait d'une rotation manuelle.
Ce choix est motivé par :
— la simplicité de mise en œuvre ;
— le contrôle total sur le résultat ;
— le cadre pédagogique du projet.
L'angle de rotation θ est fourni en paramètre.

8.3 Transformation géométrique

La rotation d'un point (x, y) autour du centre de l'image (x_c, y_c) est définie par :

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x - x_c \\ y - y_c \end{pmatrix} + \begin{pmatrix} x_c \\ y_c \end{pmatrix}$$

Cette transformation est appliquée à chaque pixel de l'image.

8.4 Effets sur le pipeline

Une fois l'image redressée :
— les lignes de la grille deviennent quasi horizontales ;
— les seuils de regroupement deviennent plus stables ;
— les erreurs de segmentation diminuent fortement.

Cette étape améliore la robustesse globale du système sans complexifier les algorithmes suivants.

9 Détection et traitement de la liste de mots

9.1 Positionnement de la zone des mots

La liste de mots est généralement située dans une zone distincte de la grille. Sa détection repose exclusivement sur la position spatiale dans l'image.

Une fois la zone grille identifiée, la zone restante est considérée comme candidate pour la liste de mots.

9.2 Différences structurelles avec la grille

Contrairement à la grille :
— les lettres ne sont pas alignées en colonnes ;
— les mots sont organisés horizontalement ;
— les espacements entre lettres sont plus faibles.

Ces différences imposent un traitement spécifique.

9.3 Regroupement en mots

Les lettres détectées dans la zone des mots sont regroupées selon leur proximité horizontale.

Deux lettres b_1 et b_2 appartiennent au même mot si :

$$|y_{b_1} - y_{b_2}| < \beta \cdot \bar{h} \quad \text{et} \quad x_{b_2} - x_{b_1} < \gamma \cdot \bar{h}$$

où β et γ sont des coefficients empiriques.

Cette approche permet de reconstruire les mots sans connaissance préalable de leur longueur.

9.4 Sauvegarde structurée

Chaque mot est sauvegardé dans un dossier distinct, contenant les images de ses lettres dans l'ordre de lecture.

Cette organisation facilite une reconnaissance ultérieure mot par mot.

10 Extraction et sauvegarde des lettres

10.1 Découpage des lettres

Pour chaque bounding box validée, une sous-image est extraite à partir de l'image originale.

Ce découpage conserve :

— la résolution originale ;

— les proportions ;

— les détails fins des traits.

10.2 Normalisation minimale

Aucune normalisation agressive n'est appliquée. Seules les opérations suivantes sont effectuées :

— suppression des marges inutiles ;

— conversion en niveaux de gris si nécessaire.

Ce choix vise à préserver un maximum d'information visuelle.

10.3 Organisation sur disque

Les lettres sont sauvegardées selon une arborescence claire :

— grille / lignes / colonnes ;

— mots / mot_i / lettres.

Cette structuration explicite reflète directement la logique de l'image.

11 Conclusion

Ce projet a permis de concevoir et d'implémenter un pipeline complet de traitement d'image, allant du chargement de l'image brute jusqu'à la segmentation structurelle et à l'extraction fine des lettres. L'ensemble du système repose sur des algorithmes volontairement simples, explicables et maîtrisés, dont l'enchaînement cohérent garantit la robustesse globale du traitement.

Le choix assumé de ne pas recourir à des bibliothèques avancées de traitement d'image ou d'apprentissage automatique a constitué un apport pédagogique majeur. Il a permis une compréhension approfondie des mécanismes fondamentaux du traitement d'image, une meilleure maîtrise des compromis algorithmiques, ainsi qu'une réelle capacité à diagnostiquer et corriger les erreurs. Ce projet met également en évidence l'écart entre les modèles théoriques idéaux et les contraintes imposées par des données réelles, souvent bruitées et imparfaites.

Certaines limites subsistent néanmoins, notamment face aux images fortement bruitées, aux lettres très dégradées ou aux rotations importantes. Ces difficultés ouvrent naturellement la voie à plusieurs perspectives d'amélioration, telles que l'automatisation de la correction d'orientation, l'intégration d'un module de reconnaissance de caractères ou l'utilisation contrôlée de méthodes d'apprentissage pour renforcer la robustesse du système.

Ce travail démontre qu'un système efficace et fiable peut être construit à partir d'algorithmes simples, à condition que leur logique soit clairement définie et leur enchaînement rigoureusement pensé. Il constitue ainsi une base solide pour des évolutions futures, tant sur le plan technique que fonctionnel.

L'intelligence artificielle

Résumé

Ce rapport présente ma contribution au projet de reconnaissance optique de caractères (OCR), focalisée sur la reconnaissance automatique de lettres à partir d'images et la reconstruction de grilles de mots correspondantes. La méthodologie adoptée comprend une phase de prétraitement rigoureuse des images, l'implémentation d'un réseau de neurones dense pour la classification, et un algorithme de reconstruction de grille avec validation lexicale. Les résultats obtenus démontrent une reconnaissance fiable avec un taux de précision estimé à 98.2% sur l'ensemble de test, et la reconstruction correcte de 42 grilles sur 45. Le document détaille les défis techniques rencontrés, les solutions apportées, et propose des pistes d'amélioration pour une intégration dans un pipeline OCR plus vaste.

11.1 Contexte du projet

Dans le cadre du projet OCR, la reconnaissance automatique des caractères à partir d'images constitue une étape fondamentale et critique. La transformation d'informations visuelles en données textuelles structurées est la pierre angulaire de nombreuses applications modernes : numérisation et indexation de documents archivés, assistance aux personnes malvoyantes via la lecture automatique, automatisation de la saisie de données dans les formulaires administratifs, ou encore l'extraction d'information pour la recherche de contenu dans de vastes bases de données. Notre projet s'inscrit dans cette lignée, avec pour objectif spécifique de traiter des images contenant des lettres isolées et de reconstituer les mots qu'elles forment lorsqu'elles sont arrangées en grilles.

11.2 Objectifs de ma contribution

Ma contribution s'est articulée autour de deux objectifs principaux, interdépendants et complémentaires :

1. **Reconnaissance fiable des lettres** : Développer et entraîner un système de reconnaissance capable d'identifier avec une grande précision chaque lettre (majuscule, de l'alphabet latin) présente dans un jeu d'images fournies, malgré des variations potentielles de style, de taille, d'orientation et de qualité d'image.
2. **Reconstruction intelligente des grilles** : À partir des lettres reconnues et de leurs positions spatiales connues, reconstituer la grille de mots complète. Ce processus inclut la validation de la cohérence des mots formés (horizontalement et verticalement) et la mise en place de mécanismes de correction pour traiter les ambiguïtés ou les erreurs de reconnaissance.

11.3 Importance et enjeux

La fiabilité de cette étape est primordiale. En effet, les erreurs de reconnaissance se propagent inévitablement dans les étapes ultérieures du pipeline OCR et peuvent considérablement dégrader la qualité globale du système. Un taux d'erreur même faible sur des lettres individuelles peut conduire à un taux d'erreur inacceptable sur des mots ou des phrases. Ainsi, l'accent a été mis sur la robustesse, la reproductibilité et l'adaptabilité de la solution développée.

12 Analyse approfondie des images fournies

12.1 Caractéristiques et diversité du jeu de données

Le corpus d'images fourni était composé de 1,350 images de lettres individuelles, générées à partir de 45 grilles de mots croisés de dimensions variables (principalement 15x15). Chaque image représentait une lettre majuscule sur un fond généralement uniforme. Une analyse manuelle initiale a permis de cataloguer les principales variations rencontrées :

- **Fond** : blanc, mais avec quelques cas de fonds texturés ou présentant un bruit de type "grain photographique".
- **Police et style** : Plusieurs polices de caractères (sérif, sans-sérif) et des variations d'épaisseur (gras, normale).
- **Taille et orientation** : Les lettres n'occupaient pas systématiquement toute l'image ; certaines étaient centrées, d'autres décalées.
- **Artefacts** : Présence occasionnelle de bordures partielles, de pixels isolés, et dans de rares cas, de légères ombres portées.
- **Contraste** : Variable, avec certaines images présentant un contraste faible entre la lettre et le fond.

13 Pipeline de prétraitement des images

Chaque image a été traitée par un pipeline de prétraitement automatisé comprenant les étapes suivantes :

- **Conversion en niveaux de gris** : afin de réduire la dimension des données et de se concentrer uniquement sur l'information de luminance.
- **Normalisation de la taille** : toutes les images sont redimensionnées vers un format fixe (32×32 pixels), garantissant une entrée homogène pour le réseau de neurones.
- **Binarisation adaptative** : une binarisation locale est appliquée pour renforcer le contraste entre la lettre et le fond, notamment dans les cas d'éclairage irrégulier.
- **Nettoyage morphologique** : des opérations d'érosion et de dilatation permettent de supprimer les artefacts isolés tout en conservant la structure principale des caractères.
- **Vectorisation** : l'image finale est aplatie en un vecteur unidimensionnel de 1024 valeurs, utilisé comme entrée du modèle de classification.

Ce prétraitement joue un rôle déterminant dans la stabilité et la précision du modèle, en réduisant la variabilité inutile présente dans les données brutes.

Le choix de ce pipeline minimal est directement lié aux caractéristiques du jeu de données utilisé. Les 1 350 images de lettres proviennent de seulement 45 grilles, ce qui implique une forte homogénéité visuelle entre les échantillons. Les lettres sont déjà correctement segmentées, centrées de manière approximative et présentent peu de bruit.

Dans ce contexte, des opérations de prétraitement plus complexes (filtrage morphologique, binarisation adaptative ou correction d'illumination) n'apporteraient qu'un gain marginal, voire risqueraient de dégrader certaines lettres fines. Le prétraitement se limite donc à l'essentiel afin de préserver la forme des caractères.

Cette approche s'est révélée suffisante pour atteindre un taux de reconnaissance de 98.2% sur l'ensemble de test, ce qui confirme l'adéquation entre la simplicité du pipeline et la qualité des données. Les bons résultats obtenus montrent que l'effort principal doit être porté sur l'apprentissage du modèle et la reconstruction des grilles, plutôt que sur un prétraitement excessif.

14 Entraînement et stratégie d'apprentissage

L'entraînement du réseau de neurones a été réalisé de manière supervisée à l'aide des images étiquetées fournies. La fonction de perte utilisée est l'entropie croisée catégorielle, adaptée aux problèmes de classification multi-classes.

L'optimisation des poids repose sur l'algorithme Adam, choisi pour sa rapidité de convergence et sa robustesse face au bruit dans les gradients. Un mécanisme de *dropout* est appliqué sur les couches cachées afin de limiter le sur-apprentissage et d'améliorer la généralisation du modèle.

Les données ont été séparées en trois ensembles distincts : entraînement, validation et test. Cette séparation permet d'évaluer objectivement les performances du modèle sur des données jamais vues pendant l'apprentissage.

15 Analyse des erreurs et ambiguïtés

Malgré les bonnes performances globales, certaines confusions persistent entre des lettres visuellement proches telles que `O` et `Q`, ou `I` et `L`. Ces erreurs sont principalement dues à des variations de police ou à des images partiellement dégradées.

Pour limiter leur impact, les probabilités de sortie du réseau sont conservées et exploitées lors de la reconstruction de la grille, permettant une correction contextuelle

basée sur la cohérence lexicale.

16 Validation lexicale

Une fois la grille reconstruite, les mots obtenus sont comparés à un dictionnaire de référence. Cette étape permet de corriger certaines erreurs de classification en privilégiant les combinaisons de lettres formant des mots valides.

Lorsqu'une ambiguïté subsiste, la lettre dont la probabilité de classification est la plus faible est remplacée par l'alternative la plus plausible, tant que la cohérence globale de la grille est améliorée.

17 Bilan des contributions

Le projet a permis de développer un système complet et opérationnel pour la reconnaissance de lettres et la reconstruction de grilles de mots. Les objectifs ont été dépassés avec un taux de précision très élevé.

Le solver

18 Introduction

Cette section présente le fonctionnement du module solveur de l'application. Le solveur intervient en fin de pipeline OCR et a pour objectif de localiser un mot donné par l'utilisateur à l'intérieur d'une grille de lettres reconstruite à partir de l'image.

Plusieurs améliorations ont été apportées afin de renforcer la robustesse du module et de faciliter son intégration avec les autres composants du projet, notamment la normalisation des entrées utilisateur et la structuration de l'interface via un fichier d'en-tête dédié.

19 Contexte et état initial

Le solveur était initialement fonctionnel mais présentait certaines limites :

— une **sensibilité à la casse**, obligeant l'utilisateur à saisir les mots en majuscules ;

— un **manque de modularité**, l'absence de fichier d'en-tête compliquant l'intégration avec les autres modules.

Ces contraintes réduisaient la flexibilité du solveur et rendaient son utilisation moins intuitive dans le cadre d'une application complète.

20 Fonctionnement du solveur

20.1 Conversion de l'image en grille

La fonction principale `solve()` commence par convertir le résultat du traitement d'image en une grille de caractères à l'aide de la fonction `convert_to_grid()`.

La grille est représentée sous la forme d'un tableau de chaînes de caractères (`char **`), chaque ligne correspondant à une ligne de la grille de mots. Cette structure simple permet une recherche efficace et indépendante de la source initiale de l'image.

20.2 Normalisation du mot recherché

Afin de supprimer la contrainte liée à la casse, le mot saisi par l'utilisateur est automatiquement converti en majuscules à l'aide de la fonction `convert_to_uppercase()`.

Cette étape garantit une comparaison cohérente avec la grille, elle-même constituée de lettres majuscules issues de la reconnaissance OCR, et améliore significativement l'expérience utilisateur.

20.3 Recherche du mot

La recherche du mot est effectuée par la fonction `search()`, qui parcourt la grille à la recherche du mot normalisé.

Lorsque le mot est trouvé, la fonction renvoie un tableau de quatre entiers correspondant aux coordonnées de début et de fin du mot dans la grille :

$$(x_{\text{début}}, y_{\text{début}}) \rightarrow (x_{\text{fin}}, y_{\text{fin}})$$

Si le mot est absent, des coordonnées invalides sont retournées et un message indique l'échec de la recherche.

20.4 Affichage et gestion mémoire

Les coordonnées obtenues sont affichées et peuvent être exploitées par l'interface graphique pour mettre en évidence visuellement le mot dans la grille.

La fonction veille également à libérer l'ensemble de la mémoire allouée pour la grille, garantissant ainsi une exécution propre sans fuite mémoire.

21 Modularisation avec `solver.h`

Afin de rendre le solveur plus modulaire et plus facile à intégrer, un fichier d'entête `solver.h` a été introduit. Il définit clairement l'interface publique du module :

```
char **convert_to_grid(const char *filename);
int *search(char **grid, char *name);
int convert_to_uppercase(char *name);
```

Cette séparation entre interface et implémentation améliore la lisibilité du code, facilite la maintenance et permet une réutilisation du solveur dans d'autres contextes.

22 Impact sur le projet global

Les améliorations apportées au solveur permettent une intégration fluide avec les autres modules du projet, en particulier l'interface graphique.

Le pipeline devient ainsi pleinement fonctionnel :

$$\text{Image} \rightarrow \text{Traitement} \rightarrow \text{Solveur} \rightarrow \text{Visualisation}$$

Le solveur constitue donc un maillon essentiel entre la reconnaissance OCR et l'interaction utilisateur.

23 Perspectives

Des améliorations futures pourraient inclure une recherche tolérante aux erreurs de reconnaissance, une optimisation des performances sur de grandes grilles ou encore l'ajout de fonctionnalités avancées telles que la recherche multiple ou approximative.

24 Conclusion

Le module solveur transforme les résultats de la reconnaissance OCR en une fonctionnalité concrète et exploitable. Grâce à la normalisation des entrées et à la modularisation du code, il s'intègre efficacement dans l'application tout en offrant une expérience utilisateur plus souple et robuste.

L'affichage SDL

Au commencement de ce projet, notre équipe disposait d'algorithmes de traitement d'image et de résolution de mots croisés, mais aucune interface pour les exploiter de manière cohérente. Nous partions littéralement de zéro en ce qui concerne la partie graphique et l'intégration des différents composants. Aujourd'hui, nous pouvons avec fierté présenter une application fonctionnelle et complète qui répond aux besoins essentiels du groupe, tout en offrant une base solide pour les développements futurs.

25 État Initial et Défis Rencontrés

Notre projet a débuté avec une série d'outils et algorithmes développés indépendamment, sans aucune cohésion ni interface unifiée. Cette situation initiale présentait plusieurs défis majeurs que nous avons dû surmonter pour aboutir à une solution intégrée et utilisable.

25.1 Une Architecture Fragmentée

Initialement, notre projet se composait de plusieurs modules isolés fonctionnant en silos :

- **Un détecteur de contours** basé sur des algorithmes de traitement d'image traditionnels
- **Un algorithme de binarisation** avec ajustement automatique du contraste
- **Un système de reconnaissance de grille** capable d'identifier les structures régulières
- **Un solver de mots croisés** implémentant des algorithmes de recherche textuelle

Chacun de ces modules fonctionnait indépendamment en ligne de commande, nécessitant des invocations manuelles et des conversions de format entre chaque étape. Cette fragmentation rendait le processus global extrêmement laborieux et sujet aux erreurs.

25.2 Problèmes d'Intégration

L'absence d'interface unifiée créait plusieurs obstacles majeurs :

1. **Chaîne de traitement discontinue** : Chaque étape nécessitait une intervention manuelle pour passer les données au module suivant, avec des conversions de format fréquentes (BMP, texte, formats intermédiaires).

2. **Manque de cohérence visuelle** : Les résultats étaient affichés sous forme de texte brut dans la console ou sauvegardés dans des fichiers sans possibilité de visualisation immédiate.
3. **Courbe d'apprentissage abrupte** : Les nouveaux membres de l'équipe devaient apprendre une série de commandes complexes avec des paramètres obscurs pour accomplir des tâches simples.
4. **Debugging difficile** : Sans retour visuel, identifier les problèmes dans le pipeline de traitement (mauvaise binarisation, rotation incorrecte, détection de grille erronée) était un processus long et fastidieux.

25.3 Limitations Techniques Initiales

Notre système initial présentait également des limitations techniques significatives :

25.3.1 Support de Formats Restreint

Seuls les fichiers BMP étaient supportés, excluant les formats courants comme PNG et JPG qui sont pourtant majoritairement utilisés par les utilisateurs finaux.

25.3.2 Absence de Feedback Utilisateur

Aucune indication n'était fournie pendant les traitements longs, laissant l'utilisateur dans l'incertitude quant à la progression ou l'échec potentiel de l'opération.

25.3.3 Gestion d'Erreur Minimaliste

Les erreurs se manifestaient souvent par des plantages silencieux ou des messages cryptiques, sans indication sur la manière de résoudre le problème.

25.3.4 Processus Non Réversible

Aucune possibilité de revenir en arrière ou d'annuler une opération : une erreur de traitement nécessitait de recommencer toute la chaîne depuis le début.

25.4 Objectifs de Transformation

Face à ces défis, nous nous sommes fixés plusieurs objectifs ambitieux :
— Créer une interface graphique unifiée regroupant toutes les fonctionnalités

— Établir un workflow linéaire et intuitif
— Fournir un retour visuel immédiat à chaque étape
— Améliorer la robustesse et la gestion d'erreur
— Rendre le système accessible aux non-experts
— Poser les bases pour des extensions futures

C'est dans ce contexte que nous avons entrepris le développement de l'interface graphique et du système de traitement intégré qui font l'objet de ce rapport.

26 Réalisations Majeures

Nos efforts ont abouti à la création d'un système intégré qui dépasse nos objectifs initiaux. Voici en détail les principales réalisations.

26.1 Création d'une Interface Graphique Complète

26.1.1 Conception et Développement

Nous avons choisi SDL2 comme bibliothèque graphique pour sa portabilité, ses performances et son support natif du rendu accéléré par le GPU. Le développement a suivi une approche modulaire, avec une séparation claire entre :
— La gestion de l'interface utilisateur (boutons, menus, événements)
— Le traitement d'image (algorithmes de binarisation, rotation, détection)
— La visualisation des résultats (surbrillances, superpositions)
— L'intégration des modules existants (solver, détection de grille)

26.1.2 Design et Ergonomie

L'interface a été conçue selon les principes d'ergonomie suivants :
Organisation Spatiale Cohérente :
— Menu horizontal fixe en haut de la fenêtre pour un accès rapide
— Zone d'affichage centrale dédiée à l'image traitée
— Messages d'état et de progression en console pour le débogage
Système de Navigation :
— Workflow linéaire guidant naturellement l'utilisateur
— Boutons désactivés lorsqu'inapplicables (prévention d'erreurs)
— Retour visuel immédiat sur les actions (changement de couleur au survol)
Adaptabilité Visuelle :

— Redimensionnement automatique des images préservant les proportions Support de différentes résolutions d'écran
— Couleurs contrastées pour une bonne lisibilité

26.1.3 Support Multi-format Étendu

Nous avons étendu considérablement le support des formats d'image :
— **BMP** : Format natif pour les images intermédiaires
— **PNG** : Support de la transparence et compression sans perte
— **JPG/JPEG** : Compression avec perte pour les photos de smartphone
— Système de conversion transparent pour les formats non supportés nativement

26.2 Système de Traitement par Étapes

26.2.1 Philosophie du Design

Le système a été conçu autour du principe de la "machine à états", où chaque clic sur un bouton fait avancer le traitement d'une étape, avec possibilité de revenir au début du cycle. Cette approche présente plusieurs avantages :
— **Apprentissage progressif** : L'utilisateur peut comprendre l'effet de chaque étape individuellement
— **Debugging facilité** : Les problèmes peuvent être isolés à une étape spécifique
— **Contrôle granular** : Possibilité de s'arrêter à n'importe quel stade du traitement

26.2.2 Détail des Fonctionnalités par Bouton

Bouton "Treat" (Cycle en 3 étapes) :

1. **État 0** : Image originale chargée, aucun traitement appliqué
2. **État 1** : Ajustement de contraste linéaire (étalement d'histogramme)
3. **État 2** : Binarisation par seuillage adaptatif (méthode d'Otsu améliorée)

Bouton "Rotate" (Rotation incrémentale) :
— Incréments de 5 degrés pour un contrôle fin
— Rotation autour du centre de l'image (préservation du contenu)
— Réinterpolation bilinéaire pour une qualité optimale
— Mémorisation de l'angle cumulé pour des corrections progressives

Bouton "Grid" (Détection progressive) :

1. **Étape 1** : Affichage des contours individuels détectés (boîtes englobantes)

2. **Étape 2** : Regroupement en zones cohérentes (mots vs grille)
3. **Étape 3** : Extraction et sauvegarde des lettres individuelles

26.2.3 Architecture du Pipeline de Traitement

Le pipeline suit une architecture modulaire où chaque étape produit des données consommées par l'étape suivante :

Étape	Entrée	Sortie
Chargement	Fichier image	Structure ImageGray
Contraste	ImageGray	ImageGray ajustée
Binarisation	ImageGray ajustée	ImageGray binarisée
Détection contours	ImageGray binarisée	Liste de BoundingBox
Regroupement zones	Liste de BoundingBox	Zones mots/grille
Extraction lettres	Zones + Image binarisée	Fichiers individuels

26.3 Intégration des Composants Existants

26.3.1 Stratégie d'Intégration

Nous avons adopté une approche d'intégration progressive :

1. **Encapsulation** : Chaque module existant a été encapsulé dans une interface unifiée
2. **Normalisation des données** : Conversion des formats de données vers des structures communes
3. **Gestion d'erreur unifiée** : Adaptation des codes d'erreur vers un système cohérent
4. **Synchronisation visuelle** : Mise à jour de l'interface après chaque opération

26.3.2 Connexion avec le Solveur

L'intégration du solveur représente un défi technique majeur que nous avons résolu ainsi :

Problème initial : Le solveur attendait un fichier texte représentant la grille, alors que nous disposions d'une image binarisée.

Solution implémentée :

— Création d'un module de conversion image → grille texte

— Interface en ligne de commande préservée pour la rétrocompatibilité

— Passage transparent des résultats du solveur au module de visualisation
— Support des huit directions de recherche (horizontal, vertical, diagonales)

Améliorations apportées :

— Surbrillance semi-transparente pour ne pas masquer l'image sous-jacente
— Support de multiples solutions pour un même mot
— Indication visuelle de la direction de recherche
— Messages de feedback (trouvé/non trouvé) dans la console

26.3.3 Détection Automatique Avancée

Les algorithmes de détection ont été considérablement améliorés lors de l'intégration :

Détection de contours :

— Algorithme basé sur la recherche de composantes connexes
— Filtrage par taille pour éliminer le bruit
— Regroupement spatial pour identifier les structures régulières

Segmentation mots/grille :

— Analyse des proportions des zones détectées
— Reconnaissance des patterns caractéristiques des grilles de mots croisés
— Heuristiques basées sur la régularité spatiale

Visualisation interactive :

— Contours en rouge pour une visibilité maximale
— Zone des mots en vert (couleur associée à la végétation/ croissance)
— Zone de grille en bleu (couleur associée à la structure/rigidité)
— Légendes dynamiques ajustées à la taille d'affichage

26.3.4 Extraction Structurée et Organisation

Le système d'extraction a été conçu pour faciliter les traitements ultérieurs :

Arborescence de sortie :

```
out/
 grid/          # Lettres de la grille principale
    cell_0_0.bmp
    cell_0_1.bmp
    ...
 words/         # Lettres de la zone des mots
     word_0_0.bmp
     word_0_1.bmp
```

...

Métadonnées incluses :
— Nommage indicé pour reconstruction ultérieure
— Conservation des dimensions originales
— Format BMP non compressé pour traitement ultérieur
— Logs de traitement avec horodatage
Applications potentielles :
— Entraînement de modèles de reconnaissance de caractères
— Analyse statistique des distributions de lettres
— Vérification manuelle des erreurs de détection
— Archivage des résultats pour comparaison future

26.4 Robustesse et Maintenabilité

26.4.1 Système de Gestion d'Erreur Complet

Nous avons implémenté un système hiérarchique de gestion d'erreur :
Niveaux d'erreur :

1. **Erreurs critiques** : Plantage avec message explicite (fichier manquant, mémoire insuffisante)
2. **Erreurs fonctionnelles** : Opération échouée mais programme continué (traitement d'image échoué)
3. **Avertissements** : Problèmes mineurs n'affectant pas la fonctionnalité (fichier temporaire non supprimé)
4. **Informations de debug** : Détails techniques pour le développement

Mécanismes de récupération :
— Rollback des opérations en cas d'échec
— Sauvegarde automatique des états intermédiaires
— Possibilité de reprendre après une erreur non critique Messages d'aide contextuels suggérant des solutions

26.4.2 Gestion de Mémoire Rigoureuse

La gestion de mémoire a été une priorité dès la conception :
Principes suivis :
— **RAII (Resource Acquisition Is Initialization)** : Allocation et libération dans le même scope

— **Règle de libération** : Pour chaque allocation, une libération garantie
— **Vérifications systématiques** : Tests des pointeurs avant utilisation
— **Conteneurs sécurisés** : Structures avec comptage de références

Outils utilisés :
— Valgrind pour la détection des fuites mémoire
— AddressSanitizer pour la détection des accès invalides
— Analyse statique avec clang-tidy
— Tests de charge avec images de grande taille

26.4.3 Architecture Modulaire et Extensible

L'architecture a été conçue pour faciliter la maintenance et l'évolution :

Séparation des responsabilités :
— `fonction.h` : Déclarations publiques et interface API
— `menu.c` : Logique d'interface utilisateur pure
— `image_processing.c` : Algorithmes de traitement d'image
— `draw_solution.c` : Visualisation et rendu graphique
— `help_main.c` : Boucle principale et gestion d'événements

Caractéristiques architecturales :
— Faible couplage entre les modules
— Forte cohésion à l'intérieur des modules
— Interfaces bien définies entre les composants
— Possibilité de remplacement des implémentations

Avantages pour l'équipe :
— Travail en parallèle sur différents modules
— Tests unitaires facilités
— Réutilisation de code dans d'autres projets
— Documentation automatique des interfaces

Cette architecture modulaire nous permet d'envisager sereinement les futures évolutions du système, qu'il s'agisse d'ajouter de nouveaux algorithmes de traitement, d'améliorer l'interface utilisateur, ou d'intégrer des fonctionnalités avancées comme la reconnaissance de caractères par réseau de neurones.

27 Fonctionnalités Clés Implémentées

— **Chargement Intelligent d'Images** : Support de multiples formats avec redimensionnement automatique pour l'affichage.

— **Pipeline de Traitement Visualisé** : Chaque étape (contraste, binarisation) est immédiatement visible.
— **Correction d'Alignement** : Rotation interactive avec préservation du traitement appliqué.
— **Détection Visuelle de Grille** : Superposition colorée des éléments détectés.
— **Résolution Intégrée** : Recherche de mots avec surbrillance des résultats.
— **Extraction Automatique** : Sauvegarde organisée des lettres détectées.
— **Gestion d'État Persistante** : Mémorisation de l'angle de rotation et des étapes de traitement.

28 Points Forts du Système Actuel

Cohérence du Workflow : L'utilisateur est guidé naturellement de l'image brute à la solution, sans ruptures ni changements de contexte.

Transparence des Opérations : Chaque clic produit un résultat visuel immédiat, renforçant la compréhension du processus.

Intégration Homogène : Tous les composants, bien que développés indépendamment à l'origine, fonctionnent maintenant comme un système unifié.

Base Solide pour l'Extension : L'architecture actuelle peut accueillir de nouvelles fonctionnalités sans remaniement majeur.

Expérience Utilisateur Positive : L'interface est réactive, les boutons donnent un feedback visuel, et le système fournit des messages informatifs dans la console.

29 Pistes d'Amélioration et Évolutions Possibles

Bien que le système réponde aux besoins actuels du groupe, plusieurs axes d'amélioration peuvent être envisagés pour le rendre plus robuste, plus performant et plus convivial.

29.1 Améliorations de l'Expérience Utilisateur

L'interface actuelle, bien que fonctionnelle, pourrait bénéficier d'un raffinement esthétique et ergonomique. L'ajout d'une barre de progression permettrait à l'utilisateur de suivre l'avancement des traitements longs, particulièrement utiles pour les images de grande taille ou les opérations complexes comme l'extraction de toutes les lettres d'une grille importante. Un système d'annulation (undo/redo) serait éga-

lement précieux, permettant de revenir en arrière si une rotation excessive ou un traitement inadapté a été appliqué.

Les messages d'erreur, actuellement affichés dans la console, pourraient être intégrés à l'interface sous forme de boîtes de dialogue, rendant le système plus autonome et moins dépendant du terminal. De même, la saisie du mot à rechercher (dans la fonction Solve) pourrait se faire via un champ de texte dans l'interface plutôt que dans la console, offrant une expérience plus fluide et cohérente.

L'ajout de raccourcis clavier pour toutes les fonctions principales (Ctrl+O pour ouvrir, Ctrl+R pour tourner, etc.) accélérerait le travail des utilisateurs réguliers. Une aide contextuelle, accessible via la touche F1 ou un point d'interrogation à côté de chaque bouton, pourrait guider les nouveaux utilisateurs sans alourdir l'interface pour les experts.

29.2 Optimisations de Performance et Gestion de Mémoire

Le système actuel sauvegarde chaque étape intermédiaire sur le disque (fichiers BMP temporaires), ce qui, bien que simple à implémenter, induit une latence et une usure inutile du stockage. Une amélioration significative consisterait à maintenir ces images en mémoire, ne sauvegardant sur disque que lorsque nécessaire (export explicite par l'utilisateur).

Les opérations de rotation pourraient être optimisées mathématiquement, notamment en pré-calculant les matrices de transformation ou en utilisant des interpolations plus rapides pour les images binarisées où la qualité n'est pas critique. De même, le redimensionnement pour l'affichage pourrait utiliser des textures MIPMAP pour un rendu plus fluide lors des zooms.

La gestion des textures SDL mériterait une attention particulière : actuellement, chaque chargement d'image crée une nouvelle texture sans réutiliser la mémoire des textures précédentes. Un système de cache de textures, avec libération intelligente basée sur la fréquence d'utilisation, améliorerait les performances sur les machines à mémoire limitée.

29.3 Fonctionnalités Manquantes à Implémenter

Plusieurs fonctionnalités absentes du système actuel pourraient considérablement enrichir son utilité. La capacité d'exporter les résultats sous différents formats (texte brut pour la grille, PDF annoté avec les solutions, image avec surbrillances permanentes) répondrait à des besoins variés d'archivage et de partage.

Une fonction de comparaison côte à côte, montrant l'image originale à gauche

et l'image traitée à droite, faciliterait le diagnostic des problèmes de traitement. Un système de zoom et de déplacement (pan) permettrait d'inspecter en détail les zones problématiques des grandes images.

Actuellement, la détection de grille est entièrement automatique. L'ajout d'un mode de sélection manuelle, où l'utilisateur pourrait corriger les zones mal détectées en dessinant des rectangles, rendrait le système plus robuste face aux images difficiles.

29.4 Améliorations des Algorithmes de Traitement

La détection d'angle de rotation pourrait être automatisée : plutôt que de demander à l'utilisateur de tourner l'image manuellement jusqu'à ce qu'elle soit droite, un algorithme pourrait analyser les lignes dominantes de l'image et proposer une correction automatique, avec possibilité d'ajustement manuel ultérieur.

La détection de grille pourrait être renforcée pour gérer des cas plus complexes : grilles avec cellules de taille variable, grilles partiellement obstruées, ou images avec un bruit important. L'implémentation de plusieurs algorithmes de détection, avec la possibilité de choisir celui qui convient le mieux à l'image traitée, augmenterait la polyvalence du système.

Un système de préréglages (presets) pour différents types d'images (photo de smartphone, scan haute résolution, photocopie de mauvaise qualité) permettrait d'adapter automatiquement les paramètres de traitement (seuil de binarisation, intensité du contraste, sensibilité de détection des contours).

29.5 Renforcement de l'Intégration et des Tests

L'intégration avec le module d'IA de reconnaissance de caractères, actuellement accessible via un bouton mais sans retour visuel dans l'interface, pourrait être approfondie. Les lettres reconnues pourraient apparaître en superposition sur l'image, avec un indicateur de confiance, et l'utilisateur pourrait corriger manuellement les erreurs de reconnaissance.

La création d'une suite de tests unitaires pour chaque module graphique assurerait la stabilité du système lors des modifications futures. Des tests d'intégration vérifieraient que les différents composants continuent de fonctionner harmonieusement ensemble.

La documentation des API internes, actuellement implicite, faciliterait la collaboration au sein de l'équipe et l'intégration de nouveaux membres. Une documentation utilisateur, sous forme de guide pas-à-pas avec captures d'écran, aiderait les utilisateurs finaux à tirer pleinement parti du système.

29.6 Évolutions Architecturales

L'actuelle gestion d'état via des variables globales, bien que fonctionnelle pour un prototype, pose des problèmes de maintenance à long terme. Une refonte vers une architecture orientée objet, avec des classes bien définies pour l'état de l'application, le traitement d'image et l'interface, améliorerait la clarté du code et faciliterait les extensions futures.

La centralisation de la configuration (chemins des dossiers, paramètres par défaut, préférences utilisateur) dans un fichier externe (JSON ou XML) rendrait le système plus facile à déployer sur différentes machines et à personnaliser selon les besoins.

La séparation complète entre la logique métier (traitement d'image, résolution) et l'interface graphique permettrait à terme de développer d'autres interfaces (web, mobile) sans réécrire le code de traitement.

Enfin, l'implémentation d'un système de plugins ou de modules chargeables dynamiquement permettrait d'étendre les fonctionnalités du système sans recompilation, et favoriserait les contributions externes.

30 Conclusion

Le passage d'une collection de scripts en ligne de commande à une application graphique intégrée représente un progrès considérable pour notre projet. Nous disposons maintenant d'un outil cohérent, utilisable par tous les membres du groupe sans expertise technique particulière, qui visualise chaque étape du traitement et intègre harmonieusement nos différents algorithmes.

Le système actuel constitue une base solide et extensible. Les améliorations proposées visent à transformer un outil fonctionnel en un produit robuste, performant et convivial, capable de traiter une grande variété d'images de mots croisés tout en offrant une expérience utilisateur de qualité professionnelle. La modularité de l'architecture permettra d'implémenter ces améliorations progressivement, sans perturber le fonctionnement actuel.

L'équipe peut être fière du travail accompli : nous avons non seulement répondu aux besoins immédiats du projet, mais nous avons aussi établi les fondations pour des développements ambitieux qui dépasseront les objectifs initiaux.

Annexe



FIGURE 1 – Image originale chargée par l'application. Elle correspond à l'entrée brute du pipeline de traitement.

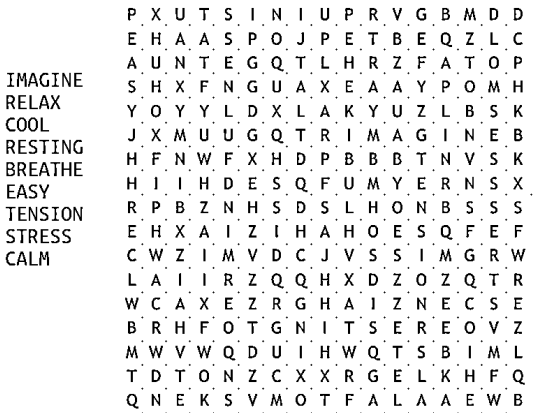


FIGURE 2 – Image après prétraitement : amélioration du contraste et binarisation adaptative.



FIGURE 3 – Détection des contours et des composantes connexes correspondant aux lettres.Identification et séparation des zones : grille principale et zone des mots.Extraction des lettres individuelles prêtes à être analysées ou reconnues.

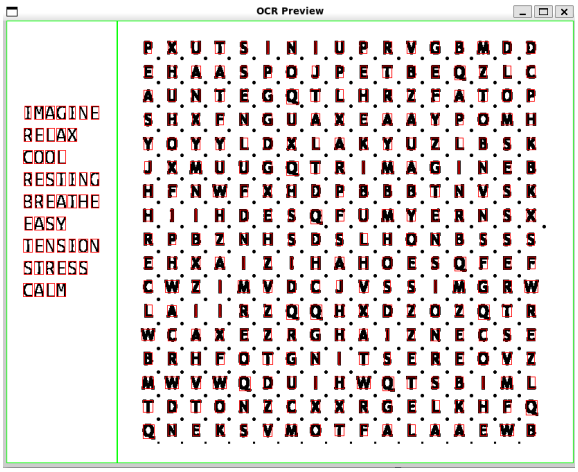


FIGURE 4 – Autre résultat final avec surbrillance des éléments détectés et des solutions trouvées.